ECON485 - Homework Assignment 1

Database Design

Part 1: Extracting Concepts

Based on the University Course Registration Rules and Regulations (Appendix 1), the comprehensive list of business objects (entities) necessary for a fully functioning registration system includes: Course, Student, Instructor, Section, Registration, Prerequisite, Department, Faculty, Academic Advisor, Schedule (Time/Room), Add/Drop Action, Withdrawal, Transcript/Grade, and Overrides.

Part 2: Selecting the Basic Concepts

For a minimal system without complex rules (no prerequisites, no time conflict checks, no history tracking), the selected core concepts are:

- **Student:** The individual registering for classes.
- **Course:** The abstract definition of an academic class.
- **Section:** A specific offering of a course in a given term.
- **Registration:** The linkage denoting a student is enrolled in a section.

Part 3: 2NF Table Design

To satisfy the Second Normal Form (2NF), the tables and their relationships are structured as follows:

- **Students:** StudentID (Primary Key), FullName
- **Courses:** CourseID (Primary Key), CourseCode, Title, Credits
- **Sections:** SectionID (Primary Key), CourseID (Foreign Key), SectionNumber, Capacity
- **Registrations:** StudentID (Foreign Key), SectionID (Foreign Key). *Composite Primary Key: (StudentID, SectionID)*

Part 4b: Limiting Rules Discussion

Limiting rules are critical in a real registration system to maintain academic integrity and ensure students progress logically through their programs. Without enforcing prerequisites, students might enroll in advanced classes without foundational knowledge, leading to high failure rates. Time conflict rules are equally important to ensure physical feasibility; a student cannot attend two lectures

simultaneously. To enforce these, the database requires additional structures such as a `Prerequisites` table (linking required courses with minimum grades) and a `Schedule` table tracking days and hours for each section. The system must query these tables before committing an INSERT operation to the Registrations table. These were excluded from Part 2 simply to establish a basic relational foundation before introducing operational complexity.

Part 4c: The Requirement to Keep History

A real registration system needs to maintain a history of all actions to support administrative audits, resolve student disputes, and accurately process financial billing. If a student drops a course, replacing the record or deleting it leaves no proof of their enrollment duration, which is problematic for transcript accuracy (e.g., assigning a 'W' for withdrawal). This requires an `ActionLog` table with fields such as `ActionID, StudentID, SectionID, ActionType (Add/Drop/Withdraw), Timestamp, and ApprovedBy`. This makes the system more complex because every state change must trigger an insert into the log, increasing data volume and requiring transactional safety. We omitted this in the basic design to focus purely on current state representation.